

Programação Competitiva

Semana 8: Estruturas de dados avançadas

Alberto Tavares Duarte Neto

08 de Outubro, 2025

Universidade de Brasília

Problema

São dados um vetor $v_1, \dots, v_n$ de $n \leq 10^5$ inteiros e um inteiro $q \leq 10^5$. Teremos q consultas, que podem ser de dois tipos:

- (i) recebe-se dois inteiros l, r com $1 \leq l \leq r \leq n$, e deve-se imprimir a soma $v_l + v_{l+1} + \dots + v_r$,
- (ii) recebe-se dois inteiros i, x com $1 \leq i \leq n$ e $-10^9 \leq x \leq 10^9$, e deve-se atualizar $v_i = x$. Não é necessário imprimir valores nesse tipo de consulta.

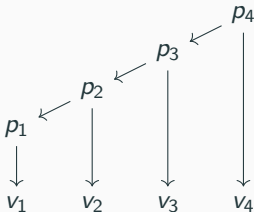
Vimos a técnica de soma de prefixos (psum). Qual é o problema dela?

Ao atualizar o valor da posição i , é necessário atualizar os valores do vetor de psum da posição i até a posição n . Em outras palavras, o valor $\text{psum}[i]$ depende dos valores das posições $1, 2, \dots, i$.

PSUM = TLE

Seja $p_1, \dots, p_n$ o vetor de soma de prefixos do vetor v . Por definição, $p_1 = v_1$ e $p_i = p_{i+1} + v_i$.

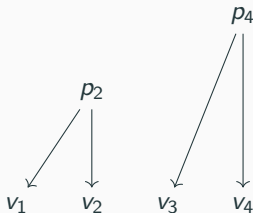
O grafo abaixo nos mostra a dependência dos valores p_i ; cada $u \rightarrow v$ mostra que o valor de u precisa do valor v para ser calculado.



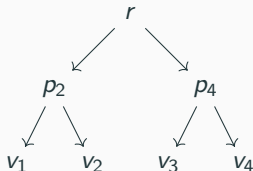
Podemos criar uma estrutura para este exemplo que minimize as dependências?

PSUM = TLE

Vamos cortar a dependência de p_3 para p_2 (isto é, cortar a aresta $p_3 \rightarrow p_2$). Neste caso, teremos $p_1 = v_1$ e $p_3 = v_3$, então desconsideraremos p_1 e p_3 . Temos:



Temos um problema: como agora $p_4 = v_3 + v_4$, não temos uma variável que tenha o valor $v_1 + v_2 + v_3 + v_4$. Adicionamos um valor $r = p_2 + p_4$ e reorganizamos a árvore:



Diminuímos a quantidade máxima de dependências de 4 para 2. Notemos que a estrutura que fizemos foi a de uma árvore binária.

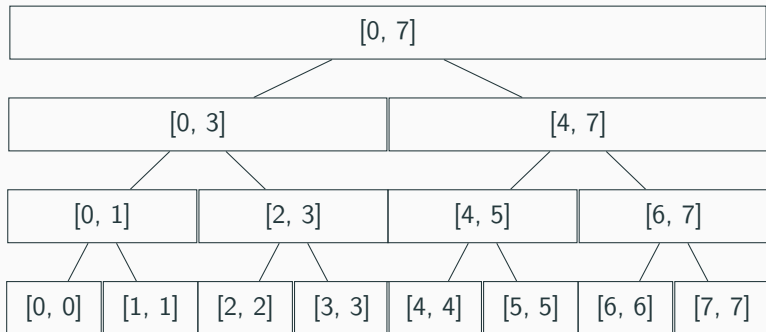
Problema: como fazer a soma $v_2 + v_3 + v_4$? Podemos soma v_2 com p_4 .

Será que podemos fazer uma estrutura similar para qualquer tamanho de vetor?

Árvore de segmentos

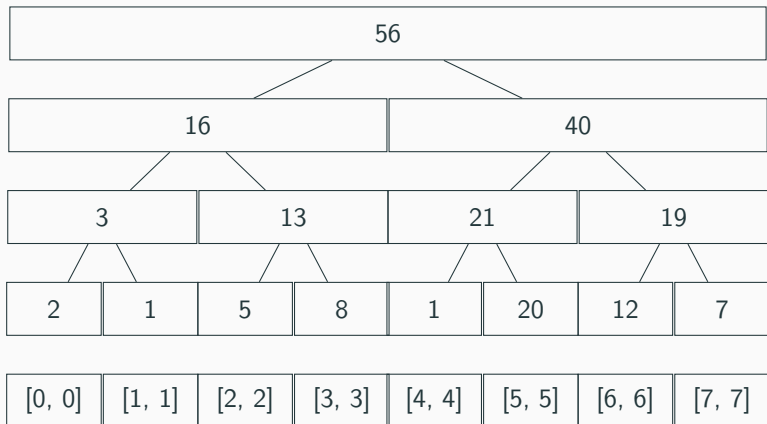
Consideremos um vetor $v_0, \dots, v_7$ de tamanho 8. Construiremos uma árvore binária de tamanho $\log_2(8)$ da seguinte forma:

- Começamos com o intervalo $[0, 7]$.
- Considerando o intervalo $[l, r]$, criamos um nó contendo a soma $v_l + \dots + v_r$. Se $l \neq r$ então dividimos o intervalo $[l, r]$ em 2 e resolvemos recursivamente para as duas metades.



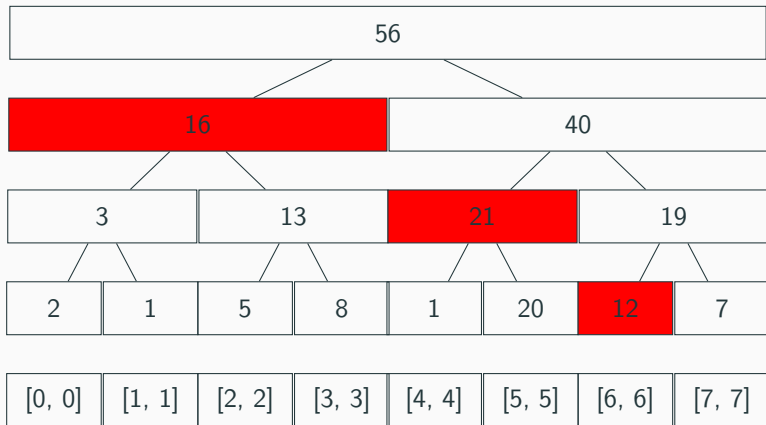
Árvore de segmentos

Consideremos o vetor $\{2, 1, 5, 8, 1, 20, 12, 7\}$. A árvore de segmentos desse vetor é:



Árvore de segmentos - soma

A soma dos elementos no intervalo $[0, 6]$ consiste na soma dos nós correspondentes aos intervalos $[0, 3]$, $[4, 5]$, $[6, 6]$, que é $16 + 21 + 12 = 49$. Os intervalos utilizados estão abaixo em vermelho:



Árvore de segmentos - soma

Seja $[l, r]$ o intervalo que queremos calcular. Supondo que a árvore de segmentos já está montada, o algoritmo para calcular a soma de l até r é:

- Começamos na raiz da árvore, que contém a soma do intervalo $[1, n]$. O algoritmo será recursivo em função do nó.
- Considere que estamos no nó correspondente ao intervalo $[lx, rx]$. Se $lx > rx$ então retornamos 0. Caso contrário, consideramos dois casos:
- Caso 1: $[lx, rx] \not\subset [l, r]$. Isso significa que existem elementos no intervalo $[lx, rx]$ que não queremos na soma. Chamamos recursivamente para os intervalos $[lx, mid]$ e $[mid + 1, rx]$, onde $mid = (lx + rx)/2$, e retornamos a soma dos retornos dessas chamadas.
- Caso 2: $[lx, rx] \subset [l, r]$. Neste caso, retornamos a soma do nó atual.

Árvore de segmentos - soma

Implementação da consulta

```
const int MAX = 4*N; // N eh o maior tamanho possivel do vetor
int t[MAX]; // armazena os valores em cada no

// [l, r] representa o intervalo que queremos calcular
// x representa o INDICE do no atual
// [lx, rx] representa o intervalo do no de indice x
int query(int l, int r, int x, int lx, int rx) {
    if(l <= lx and rx <= r) return t[x];
    if(r < lx or rx < l) return 0;

    int mid = (lx+rx)/2;
    int s1 = query(l, r, 2*x, lx, mid); // esquerda -> 2*x
    int s2 = query(l, r, 2*x+1, mid+1, rx); // direita 2*x+1

    return s1 + s2;
}
```

Árvore de segmentos - soma

Proposição

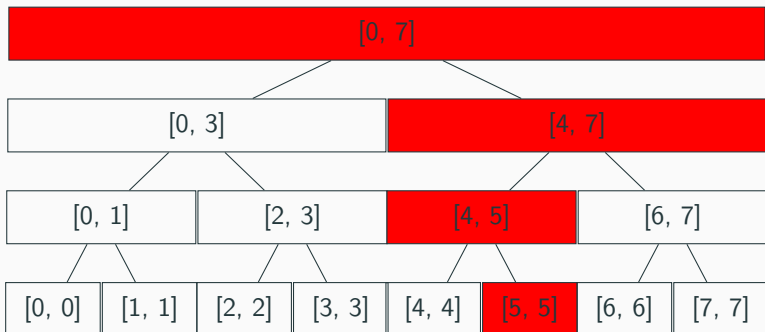
Para calcular a soma de um intervalo $[l, r]$, é necessário escolher apenas $2 \log(n)$ vértices da maneira ótima, onde n é a quantidade de nós na árvore.

Ideia da prova. Podemos provar por indução. Consideremos a menor altura a ser escolhida. Se escolhermos 3 vértices consecutivos, ou os dois primeiros ou os dois últimos terão o ancestral comum, e bastaria escolher esse ancestral, que tem a soma igual a dois dois filhos. Não escolheremos vértices não consecutivos pela continuidade do intervalo $[l, r]$.

Considemos o nível $k + 1$. Por indução, escolhemos apenas 2 vértices do nível k . Seja u um destes vértices e p_u seu ancestral. Seja v o filho de p_u diferente de u . O vértice v tem altura k , e escolheremos no máximo um de seus filhos, que terá altura $k + 1$. Pela continuidade do intervalo, os únicos candidatos da altura $k + 1$ vem deste processo.

Árvore de segmentos - atualização

Vamos atualizar o valor v_i para um inteiro x . Devemos atualizar todos os nós que dependem da posição i . Precisamos modificar apenas os ancestrais do vértice correspondente ao intervalo $[i, i]$. Como a altura da árvore é $\log_2(n)$, então precisamos modificar apenas $\log_2(n)$ nós. Por exemplo, ao atualizar $i = 5$, precisamos atualizar os nós em vermelho abaixo:



Árvore de segmentos - soma

Seja i o índice a ser atualizado, e val o novo valor de v_i .

- Começamos na raiz da árvore, que representa o intervalo $[1, n]$. O algoritmo será recursivo em função do nó.
- Considere que estamos no nó correspondente ao intervalo $[lx, rx]$. Apenas um dos filhos representa um intervalo contendo i . Se $i \in [lx, mid]$ então chamamos recursivamente para o filho da esquerda, caso contrário, chamamos recursivamente para o filho da direita.
- No final de ambos os casos, o valor de um dos filhos será atualizado. Isso significa que o valor do nó atual deverá ser atualizado (será a soma dos valores dos filhos).

Árvore de segmentos - soma

Implementação da atualização

```
// i representa o índice do elemento do vetor a ser atualizado
// val representa o novo valor do elemento
// x representa o INDICE do nó atual
// [lx, rx] representa o intervalo do nó de índice x
void update(int i, int val, int x, int lx, int rx) {
    if(lx == rx) {
        t[x] = val;
        return;
    }
    int mid = (lx+rx)/2;
    // chama recursivo para atualizar os filhos
    if(i <= mid) update(i, val, 2*x, lx, mid);
    else update(i, val, 2*x+1, mid+1, rx);

    // com os filhos atualizados, atualizamos o nó atual
    t[x] = t[2*x] + t[2*x+1];
}
```

Problema

São dados dois inteiros n e q , com $n, q \leq 10^5$. Considere que inicialmente temos n conjuntos $\{1\}, \{2\}, \dots, \{n\}$. Serão feitas q consultas, podendo ser de dois tipos:

- 1 $i j$ - faça a união do conjunto que contém i com o conjunto que contém j .
- 2 $i j$ - responda se i e j estão no mesmo conjunto.

Vamos primeiro analisar o algoritmo força bruta. Declaramos um vector com n sets, com o i -ésimo set contendo inicialmente apenas o inteiro i . Para cada inteiro i , armazenamos o índice do conjunto que o contém.

Para cada operação de união, inserimos todos os elementos do set de índice i no set de índice j (e atualizamos o índice do set destes elementos), e zeramos o set i .

Para cada operação do tipo 2, apenas verificamos os índices do conjunto que as contém são iguais. Complexidade total: $O(nq \log n)$.

Small to large

O que causa a complexidade ruim da força bruta é a grande quantidade de inserções que devemos fazer.

Consideremos a seguinte heurística: se o set de índice i for menor, então inserimos todos os elementos do set i no set j ; caso contrário, inserimos todos os elementos do set j no set i .

Proposição

Com a heurística acima, serão feitas no máximo $n \log n$ inserções.

Prova. Consideremos um elemento x qualquer. Afirmamos que este elemento vai mudado de conjunto no máximo $\log n$ vezes. De fato, se ele foi inserido em outro set, o set de destino tem tamanho maior ou igual ao set de origem. Neste caso, a união terá pelo menos o dobro do tamanho do set de origem. Após $\log n$ inserções deste elemento, necessariamente o conjunto que o contém tem n elementos, não sendo mais possível novas inserções de x .

Considerando que qualquer elemento vai ser inserido no máximo $\log n$ vezes, serão feitas no máximo $n \log n$ inserções.

Podemos implementar uma estrutura de dados melhor, sem a necessidade da utilização de sets. É uma estrutura de dados que chamamos de DSU (Disjoint set union).

Ideia principal. No lugar de manter vários sets, cada conjunto terá um único representante. Isto é implementado de forma que cada nó tenha um pai, e o representante seja a raiz da árvore gerada. Assim, as operações são:

- União: sejam a o representante do menor conjunto e b o representante do maior conjunto. Definiremos o novo pai de a sendo b .
- Procura: para saber se a e b estão no mesmo conjunto, basta ver se possuem o mesmo representante.

Pela heurística de *small to large*, a partir de cada elemento, é necessário visitar até $\log n$ vértices até chegar na raiz da árvore.

No DSU, podemos fazer uma segunda otimização. Ao procurar um representante de um elemento, o faremos apontar diretamente para a raiz, para que próximas consultas sejam melhores. O algoritmo para encontrar o representante de um elemento u é:

- Se $u = \text{pai}[u]$, então retornamos u .
- Se $u \neq \text{pai}[u]$, então recursivamente encontramos o representante do pai de u , e definimos o pai de u como este representante.

Pode ser provado que a complexidade final, com esta otimização, é $O((q + n)\alpha(n))$, onde $\alpha(n)$ é a inversa da função de Ackermann. Na prática, podemos considerar que a complexidade é $O(q + n)$.

Implementação da busca binária

```
vector<int> parent(n+1), sz(n+1, 1);  
for(int i = 0; i < n+1; i++) parent[i] = i;  
  
int find(int u) {  
    if(u == parent[u]) return u;  
    return parent[u] = find(parent[u]);  
}  
  
void join(int a, int b) {  
    a = find(a); b = find(b);  
    if(a == b) return;  
    if(sz[a] < sz[b]) swap(a, b);  
    parent[b] = a;  
    sz[a] += sz[b];  
}
```